# When a Self-Improving Loop Improves Itself and When It Drifts: Toy-Scale Experiments

**Author:** Eric Lam (independent researcher, Toronto)

## Abstract

A model that generates candidates, checks them, and retrains on what passed can improve, or it can drift toward its own errors. We study this on toy tasks and small models. Our working claim is that the loop stays sound when correctness comes from an external check, when verified items accumulate in an archive instead of being refit and forgotten, and when exploration is built into the proposals. On an eight-bit task with a known answer set, rules that train on their own outputs or on a learned judge drift or stop confidently at low accuracy, while an archive with an external check reaches the full correct set. In neural runs, a character-level GRU on arithmetic and number words, with a Python evaluator or a dictionary as teacher, did not learn the underlying rules, but student plus teacher solved 85% of held-out arithmetic targets at 100 attempts. Each result comes from one fixed seed on small tasks, so we report them as observations, not general laws.

## 1. Introduction

Self-improvement needs no extra labels: the model proposes, something checks, and the model learns from what passed. The obvious failure is drift. A rule that refits to its own samples can collapse onto a few outputs, reward a judge that errs in predictable places, or sharpen onto one answer that scores well but lacks diversity.

The working theory, written up in `THEORY.md`, names four failure modes: self-reference (the judge is a function of the learner), drift (refitting to finite samples of one's own output), Goodhart (a judge learned from few labels has errors the learner finds), and degenerate optimum (maximising expected reward yields a single answer). It proposes one rule that addresses all four: keep a monotone archive of verified items, propose from the archive and from mutations, answer from the archive, and stop when the discovery rate plateaus.

The experiments isolate each failure mode in a controlled comparison and then test the rule against a learned verifier, a neural student, and learners that invent their own structure.

## 2. Method

**The loop.** Each round the learner samples candidates, an external verifier V marks each correct or not, and an update rule maps the kept candidates to the next distribution.

**Update rules.** On the toy task we compare thirteen rules: rejection-sampling fine-tuning (`rft`), a trust-region mix (`trust_region`), REINFORCE with and without an entropy bonus (`reinforce`, `reinforce_entropy`), greedy policy improvement (`greedy`), self-training on own outputs (`self_distill`,

`self_judge`), learned-judge rules (`reward_model`, `unified_proxy`), evolutionary search (`evolve`), data anchoring (`anchor`), smoothing (`smooth`), and the proposed rule `Unified` (model 12). `Unified` proposes samples, mutations of samples, and a small uniform set; adds the verified proposals to a monotone archive A; answers from the uniform distribution over A; and reports "thinks" as one minus the fraction of verified items that were new this round. Runs stop when "thinks" changes by less than a fixed epsilon.

**Follow-ups.** The verifier is a learned logistic model that receives two oracle labels per round, chosen by random auditing, a frozen proxy, or the learner's own uncertain outputs. Learners are a 256-parameter table, an 8-parameter independent-bit model, and a 72-parameter counting model. The learner also chooses its own tasks under five reward rules.

**Neural and non-neural learners.** The neural student is a character-level GRU (32-dimensional embeddings, 128 hidden units), trained with Adam at learning rate 3e-3 on its notebook. Non-neural learners are a symbol-inventing context model (`selfloop6.py`) and a program learner over a stack-machine language (`selfloop7.py`, `selfloop8.py`).

**Seeds.** Each script runs one fixed seed (0, 1 or 2 depending on the file).

## 3. Experiments and Results

### 3.1 Toy task

The correct set is the 70 eight-bit strings with exactly four ones. A learned distribution over all 256 strings can represent it exactly.

| Rule | Result | Verdict |
|---|---|---|
| `Unified` (model 12), true verifier | Stops at round 5 to 7 "on every seed" (per `THEORY.md`); 100% correct at the stop | Supported (toy) |
| `Unified`, learned verifier (`unified_proxy`) | Stops at round 5, fully confident, 25% correct | Negative |

The other rules fail as the theory predicts. `THEORY.md` gives no per-rule numbers for them.

### 3.2 Growing the verifier

Auditing two items per round with an exact oracle, targeted at the learner's own proposals, turned a 72%-accurate proxy into an exact one within 190 labels, and filled the archive twice as fast as random auditing. A frozen proxy never improved.

### 3.3 Learner size and inductive bias

| Learner | Result | Verdict |
|---|---|---|
| 256-parameter table | Memorises the archive and never leaves it | Negative |
| 8-parameter independent-bit model | Proposes at chance indefinitely (wrong bias) | Negative |
| 72-parameter counting model | About 50 examples give 90% correct proposals; archive complete in 3 rounds | Supported (toy) |

Bias mattered more than size.

### 3.4 Learner-chosen tasks

The learner picks from 72 tasks, 28 of them impossible. Under a reward for failure, 77% of the budget went to impossible tasks. A learnability reward, `s(1-s)`, matched a uniform curriculum on competence but cannot tell hard from impossible. Adding an executor validity check (the Absolute Zero recipe) removed the waste on impossible tasks, per `THEORY.md`; no number is given.

### 3.5 Arithmetic with a neural student

The student writes an expression equal to a target from 0 to 99, checked by a Python evaluator. Twenty targets are held out.

| Variant | Train targets (student alone) | Held-out (student alone) | Held-out (student plus teacher, pass@100) | Notebook size |
|---|---|---|---|---|
| Verified, no exploration | 0.78 | 0.01 | 0.15 | 441 |
| Verified plus exploration | 0.31 | 0.04 | 0.85 | 11,500 |
| Self-judge | 0.18 | 0.01 | 0.30 | 451 |
| No archive | 0.45 | 0.05 | 0.10 | 425 |

Without exploration the student memorised the seed pairs and the discovery signal correctly reported nothing new by round 5. With exploration the notebook grew monotonically, the student's own held-out precision stayed between 0.01 and 0.05, and its diversity rose to 98% distinct outputs. The diverse, mostly wrong student plus the teacher solved most held-out targets; the self-judge and no-archive variants did not.

### 3.6 English number words

A dictionary is the teacher. The student was taught 40 of the 80 training numbers and had to compose the rest, checked before entering the notebook.

| Variant | Notebook after 30 rounds | Untaught numbers (student alone) | Held-out (student alone) | Held-out pass@20 |
|---|---|---|---|---|
| Verified | 72 / 80 | 0.80 | 0.60 | 0.80 |
| Self-judge | 44 / 80 | 0.10 | 0.15 | 0.30 |

The verified student composed 32 of the 40 untaught numbers correctly. Its held-out errors fall where English is irregular: it wrote 15 as "fifty-one" and never found 10, 11, 13, 14, 17, 18 or 19. The self-judge student's confidence rose from 0.50 to 0.96 while it was stuck at its 40 taught words.

### 3.7 Thinking in a private language

The student may write up to eight symbols from a private alphabet before answering. Pads that led to a correct answer were kept. Tests swapped in shuffled or random pads and tried to read the pads as a code.

| Variant | Held-out | Own / shuffled / random pads | Distinct pads |
|---|---|---|---|
| Free pad, monotone notebook | 0.05 | 0.06 / 0.06 / 0.04 | 2689 |
| Free pad, latest per task | 0.05 | 0.05 / 0.05 / 0.04 | 1957 |
| Free pad, sampled at T=0.5 | 0.05 | 0.04 / 0.04 / 0.05 | 1811 |
| Reader sees only the pad, 10^8 messages | 0.03 | 0.02 / 0.03 / 0.03 | 1505 |
| 100 messages (2 symbols of 10) | 0.03 | 0.03 / 0.02 / 0.03 | 99 |
| No pad | 0.13 | n/a | n/a |
| Fixed pad "aaaaaaaa" | 0.23 | n/a | n/a |

Shuffled and random pads scored the same as the student's own, so the pad carried no content. A dictionary built from the notebook agreed with the reader on 0.00 to 0.02 of pads in the free-pad and reader-only variants. In the 100-message variant the agreement was 0.38 (0.37 after 100 rounds). `THEORY.md` describes writer and reader as never agreeing on a code; we could not reconcile that wording with the 0.38 figure, so we report the number without interpretation. Constant filler improved held-out accuracy from 0.13 to 0.23.

### 3.8 Non-neural learners: word streams and recursion

A symbol-inventing learner was trained on a stream of hidden words from lexicon A, then B, then A again. Bits per letter fell in each phase (A: 2.41 to 1.98; B: 3.79 to 1.69; A again: 2.15 to 1.96). All 16 hidden words became symbols, and 110 of the 133 invented symbols were whole hidden words or runs of them. Sampled text was fully covered by hidden words (1.00, against 0.26 for random letters).

Connected to a small world with four actions, learning progress reached 0.92 bits per letter at turn 600 and stopped at turn 342. Raw surprise fell into the noisy-TV failure: 92% of turns went to the noise action by turn 600, with world cost 1.81 bits per letter. Unplugged at turn 200, the learner invented 27 more symbols from its own output, and its knowledge of the world did not change.

### 3.9 Programs and libraries

A stack-machine language with four hidden macros was tested at depths 1 to 3, with 20 episodes per depth and 5000 samples per episode.

| Mode | Solved, depth 1 / 2 / 3 | Samples per episode, depth 1 / 2 / 3 |
|---|---|---|
| Full (language plus invented words) | 0.90 / 0.80 / 0.35 | 2041 / 4080 / 4876 |
| No words | 0.60 / 0.30 / 0.05 | 3336 / 4829 / 5000 |
| Random tokens | 0.85 / 0.20 / 0.00 | 4832 / 4845 / 4942 |

The full mode invented 18 words, including all four macros. At depth 1 the random baseline is close (0.85 against 0.90, one episode out of 20); the advantage appears at depths 2 and 3. A two-level version (macros, then phrases built from macros) showed the full mode solving all four blocks (1.00 each), the no-words mode solving 0.25 of the two-phrase block, and the random mode solving 0.90, 0.45, 0.10 and 0.95 of the four blocks. On the final macro block the full mode needed 449 samples per episode, against 1183 on the first, which shows retention.

## 4. Discussion

Three of the four predicted failure modes appeared. A self-judge stopped confident and wrong. A learned judge capped the loop at its own accuracy. The program learner trained only on its own successes collapsed onto the first solution pattern it found (per `THEORY.md`; no numbers given). `THEORY.md` attributes the degenerate-optimum failure to the greedy and reinforce rules, but gives no numbers for them.

The archive plus external check worked where it could. The toy loop reached full accuracy and stopped on a discovery signal. The neural notebook grew without collapse. The neural student did not learn arithmetic in the sense of generalising to new targets; it learned spelling, and it was useful as a proposer for a checker. English shows the same pattern, with errors where English is irregular.

Exploration was required. Without it the arithmetic student stopped at the seed set; with it the notebook grew to 11,500 verified answers. The non-neural random-token baseline was competitive at depth 1 but fell away at depths 2 and 3.

The thinking experiment is the clearest negative result. Free-form pads did not carry a message. `THEORY.md` suggests outcome-only selection at low hit rates is too weak to coordinate a writer and reader, and that a grade per step, as an executable trace provides, may be what is missing.

## 5. Limitations

- All tasks are small and the verifiers are exact. Nothing was tested with a real language model or a real checker such as a test suite or proof assistant.
- Each script runs one fixed seed with no error bars. `THEORY.md` says the toy result holds "on every seed", which the code does not show.

- The source folder contains no printed output logs. Numbers are taken from `THEORY.md`, which summarises printed runs; the scripts were not re-run for this paper.
- `THEORY.md` describes the arithmetic student as tested at two model sizes; the code shows only the 128-unit configuration. `THEORY.md` also says the student's held-out precision stayed "near 4%" in every variant, which the table does not show.
- Several claims are qualitative in the source: the narrowing of the learner's curriculum under a success reward, and the Absolute Zero validity check.
- Primitives, merge thresholds, exploration shares and the "shorter wins" criterion are hand-fixed.
- The almost-sure convergence result in `THEORY.md` is stated, not proved in the files.

## 6. Conclusion

At toy scale, a self-improvement loop that accumulates verified items, checks them against an external verifier, and explores through its proposals improves without drifting, and stops on an honest signal. The same loop with a learned judge stops confident and wrong; with a self-judge it does not improve. Exploration is necessary, and inductive bias matters more than size. The loop is only as good as its access to ground truth. Whether these results hold at larger scale, with real verifiers and multiple seeds, is untested.

## References

Named in the source material (`THEORY.md`) only; full citations were not given there, so none are supplied.


- AlphaEvolve (program database; named as an example system).
- Darwin Gödel Machine (agent lineage as archive; named as an example system).
- STaR and expert iteration (replay of verified traces; named as example methods).
- Shumailov (training on one's own outputs; model collapse).
- DreamCoder (library learning; named in a comparison).
- Absolute Zero (self-play with an executor validity check).
- RLHF and RLVR (reference points for learned-reward and verifiable-reward training).
- "Dot by dot" (filler tokens as extra computation).
- Noisy-TV problem (surprise-seeking agents).
- Re-Pair and PPM (pair merging; prediction by partial matching).
- Minimum description length (stopping criterion for merges).